System Call Analysis Report: `sample_binary`

Automated headless binary decompilation via Ghidra · Analysis date: 2026-10-12 22:23:25 UTC · Mode: Ghidra Headless (No GUI) | PyGhidra in-process

TL;DR

Suspicious confidence: medium

The program can both open network connections and start other programs, a combination common in droppers, backdoors and remote shells.

Rule-based assessment from the detected system calls, not a malware scan; details follow.

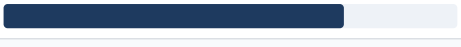
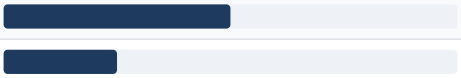
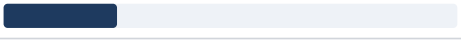
Target Binary Information

File Name	`sample_binary`
Absolute Path	`freida/samples/sample_binary`
File Format	macOS Mach-O Binary
Architecture	`AARCH64:LE:64:AppleSilicon`
File Size	33.52 KB (34,320 bytes)
SHA-256	`c0ff6f7518d5219d72f69c6e92f8e0ed240b44ac4dd0aa45f6efe6e1797b2e45`
MD-5	`e0cbdec65ec18790e069f12205c82e93`
Ghidra Program Name	`sample_binary`

Analysis Dashboard

Metric	Count	Details
Functions decompiled	**6**	Analyzed via Ghidra Decompiler
Syscall invocations	**13**	Detected across all functions & imports
Unique system calls	**12**	Distinct system call primitives
Critical severity	**0**	Injection, exec, arbitrary memory
High severity	**4**	Process spawns, page protection, privileges
Medium severity	**4**	Network sockets, file creation/deletion, IPC
Low / info severity	**5**	File reads, clocks, system telemetry

System Call Categories

Category	Count	Distribution
File I/O	4	
Process & Execution	3	
Memory Management	3	
Networking & Sockets	2	
Direct Syscall	1	

Security & Behavioral Observations

- **Direct Kernel Entry:** Raw syscall instructions that bypass libc wrappers in: `_perform_direct_syscall`.
- **Process Execution Capabilities:** Routines capable of spawning or altering processes: `execve, exit, fork`.

- **Dynamic Memory & Page Protection:** Target manages memory allocations and page permissions: `mmap, mprotect, munmap`.
- **Network Communication:** Target initializes network sockets or connections: `connect, socket`.
- **Filesystem Operations:** Target accesses, modifies, or inspects files: `close, open, write`.

System Calls Inventory

Syscall / API	Category	Severity	Function / Address	Line	Type	Arguments / Context
`direct_syscall`	Direct Syscall	HIGH	`_perform_direct_syscall 1000004b0`	5	Direct Syscall	—
`open`	File I/O	MEDIUM	`_file_io_operations` call @ `100000540`	7	Libc Wrapper	`"/tmp/output.dat",0x201`
`write`	File I/O	LOW	`_file_io_operations` call @ `100000564`	9	Libc Wrapper	`iVar1,"Data payload",0xc`
`close`	File I/O	INFO	`_file_io_operations` call @ `10000056c`	10	Libc Wrapper	`iVar1`
`socket`	Networking & Sockets	MEDIUM	`_network_operations` call @ `1000005a8`	10	Libc Wrapper	`2,1,0`
`connect`	Networking & Sockets	HIGH	`_network_operations` call @ `1000005d0`	18	Libc Wrapper	`iVar1,&sStack_28,0x10`
`close`	File I/O	INFO	`_network_operations` call @ `1000005d8`	19	Libc Wrapper	`iVar1`
`fork`	Process & Execution	MEDIUM	`_process_execution` call @ `100000624`	12	Libc Wrapper	—
`execve`	Process & Execution	HIGH	`_process_execution` call @ `100000660`	17	Libc Wrapper	`"/bin/ls",&local_30,(char **)0x0`
`exit`	Process & Execution	INFO	`_process_execution` call @ `100000668`	19	Libc Wrapper	`0`
`mmap`	Memory Management	MEDIUM	`_memory_manipulation` call @ `1000006c0`	7	Libc Wrapper	`0,0x1000,3,0x1002,0xffffffff,0`
`mprotect`	Memory Management	HIGH	`_memory_manipulation` call @ `1000006d4`	8	Libc Wrapper	`uVar1,0x1000,7`
`munmap`	Memory Management	LOW	`_memory_manipulation` call @ `1000006e0`	9	Libc Wrapper	`uVar1,0x1000`

Decompiled Evidence & Callsite Context

Function: `_perform_direct_syscall`

Detected system calls: `direct_syscall`

Line 5 · `direct_syscall` · Direct Syscall

```
CallSupervisor(0);
```

Full decompiled function: `_perform_direct_syscall`

```
void _perform_direct_syscall(void)

{
  CallSupervisor(0);
  if (*(long *)PTR___stack_chk_guard_100004000 - *(long *)PTR___stack_chk_guard_100004000 != 0
      ) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail(*(long *)PTR___stack_chk_guard_100004000 -
                     *(long *)PTR___stack_chk_guard_100004000);
  }
```

```
  return;
}
```

Function: `_file_io_operations`

Detected system calls: `open`, `write`, `close`

Line 7 · `open` · File I/O

```
iVar1 = _open("/tmp/output.dat",0x201);
```

Line 9 · `write` · File I/O

```
_write(iVar1,"Data payload",0xc);
```

Line 10 · `close` · File I/O

```
iVar1 = _close(iVar1);
```

Full decompiled function: `_file_io_operations`

```
int _file_io_operations(void)

{
  int iVar1;

  iVar1 = _open("/tmp/output.dat",0x201);
  if (-1 < iVar1) {
    _write(iVar1,"Data payload",0xc);
    iVar1 = _close(iVar1);
  }
  return iVar1;
}
```

Function: `_network_operations`

Detected system calls: `socket`, `connect`, `close`

Line 10 · `socket` · Networking & Sockets

```
iVar1 = _socket(2,1,0);
```

Line 18 · `connect` · Networking & Sockets

```
_connect(iVar1,&sStack_28,0x10);
```

Line 19 · `close` · File I/O

```
iVar1 = _close(iVar1);
```

Full decompiled function: `_network_operations`

```
void _network_operations(void)

{
  int iVar1;
  sockaddr sStack_28;
  long local_18;

  local_18 = *(long *)PTR____stack_chk_guard_100004000;
  iVar1 = _socket(2,1,0);
  sStack_28.sa_family = '\x02';
  sStack_28.sa_data[0] = '\x1f';
  sStack_28.sa_data[1] = -0x70;
  sStack_28.sa_data[2] = '\0';
  sStack_28.sa_data[3] = '\0';
  sStack_28.sa_data[4] = '\0';
  sStack_28.sa_data[5] = '\0';
  _connect(iVar1,&sStack_28,0x10);
  iVar1 = _close(iVar1);
  if (*(long *)PTR____stack_chk_guard_100004000 - local_18 != 0) {
```

```
                    /* WARNING: Subroutine does not return */
    ___stack_chk_fail(*(long *)PTR____stack_chk_guard_100004000 - local_18,iVar1);
  }
  return;
}
```

Function: `_process_execution`

Detected system calls: `fork`, `execve`, `exit`

Line 12 · `fork` · Process & Execution

```
pVar1 = _fork();
```

Line 17 · `execve` · Process & Execution

```
_execve("/bin/ls",&local_30,(char **)0x0);
```

Line 19 · `exit` · Process & Execution

```
_exit(0);
```

Full decompiled function: `_process_execution`

```
void _process_execution(void)

{
  pid_t pVar1;
  char *local_30;
  undefined *puStack_28;
  undefined8 local_20;
  long local_18;

  local_18 = *(long *)PTR____stack_chk_guard_100004000;
  pVar1 = _fork();
  if (pVar1 == 0) {
    puStack_28 = PTR_s__la_100004070;
    local_30 = PTR_s__bin_ls_100004068;
    local_20 = DAT_100004078;
    _execve("/bin/ls",&local_30,(char **)0x0);
                    /* WARNING: Subroutine does not return */
    _exit(0);
  }
  if (*(long *)PTR____stack_chk_guard_100004000 - local_18 != 0) {
                    /* WARNING: Subroutine does not return */
    ___stack_chk_fail(*(long *)PTR____stack_chk_guard_100004000 - local_18);
  }
  return;
}
```

Function: `_memory_manipulation`

Detected system calls: `mmap`, `mprotect`, `munmap`

Line 7 · `mmap` · Memory Management

```
uVar1 = _mmap(0,0x1000,3,0x1002,0xffffffff,0);
```

Line 8 · `mprotect` · Memory Management

```
_mprotect(uVar1,0x1000,7);
```

Line 9 · `munmap` · Memory Management

```
_munmap(uVar1,0x1000);
```

Full decompiled function: `_memory_manipulation`

```
void _memory_manipulation(void)

{
```

```
  undefined8 uVar1;

  uVar1 = _mmap(0,0x1000,3,0x1002,0xffffffff,0);
  _mprotect(uVar1,0x1000,7);
  _munmap(uVar1,0x1000);
  return;
}
```

Methodology & Tooling

This report was generated using **Freida** in full **Headless Mode**:

- **Decompilation Engine:** Ghidra Decompiler (DecompInterface) driven in-process via PyGhidra.
- **Headless Execution:** Fully automated headless analysis without the Ghidra graphical user interface.
- **Detection Scope:** Direct syscall instructions (syscall, sysenter, int 0x80, svc #0), POSIX / libc wrappers, Windows Native NT and Win32 APIs, and macOS Mach traps in native executables (ELF, PE, Mach-O).